Git personal notes

Figure 1: SmartOS logo as example figure for typst document

See Figure 1 for details.

Contents

Git personal notes 1
Git digging in non-branch commits 3
If the repository itself is corrupted 3
Alias copied from here: https://stackoverflow.com/questions/2092810/browse-orphaned-commits-in-git 3
Never git pull 3
Git workflow for trees 3
1. Create bare repository 3
2. Add initial worktree with sample data 3
3. Create feature worktree from main 3
4. Work on feature 3
5. Merge feature into main 3
6. Clean up (optional) 3
Common use cases 4
git local host or public server with ssh 4
Hooks 6
Client-Side Hooks 6
Server-Side Hooks 6
Original documentation 6
Client-Side Hooks 7
Email Workflow Hooks 7
Server-Side Hooks 8
Git repository types 9
AI(Artificial Intelligence) vs HI(Human Intelligence?) Or AI with HI? 9
Key Differences 9
When to Use Which 9

Figures

List of Figures

Figure 1 SmartOS logo as example figure for typst document 1
Figure 2 This is what AI answered when asking about the pros and cons of `git --bare` 10

Tables

List of Tables

Table 1 The pros and cons of `git --bare` 9

Git digging in non-branch commits

```
git reflog --all
git log --graph --reflog
git graph --reflog
gitk --reflog
gitk --reflog --date-order --all
```

If the repository itself is corrupted

```
git fsck
# Save the output to a file since it might take a minute.
git fsck --unreachable > unreachable.txt
# Note unreachable.txt now includes all unreachable blobs, trees, and commits.
cat unreachable.txt | grep commit
```

Alias copied from here: https://stackoverflow.com/questions/2092810/browse-orphaned-commits-in-git

```
alias orphank = "!gitk --all --date-order 'git reflog | cut -c1-7'"&
```

Never git pull

```
git fetch
git merge
```

Git workflow for trees

1. Create bare repository

```
mkdir ~/project.git && cd ~/project.git
git init --bare
```

2. Add initial worktree with sample data

```
git worktree add -b main ~/project/main
cd ~/project/main
echo "# My Project" > README.md
git add README.md
git commit -m "Initial commit"
cd -   # back to bare repo
```

3. Create feature worktree from main

```
git worktree add -b feature-x ~/project/feature-x main
```

4. Work on feature

```
cd ~/project/feature-x
echo "print('feature')" > feature.py
git add feature.py
git commit -m "Add feature X"
```

5. Merge feature into main

```
cd ~/project/main
git fetch . feature-x
git merge feature-x
```

6. Clean up (optional)

```
git worktree remove ~/project/feature-x
git branch -d feature-x# 1. Create bare repository
mkdir ~/project.git && cd ~/project.git
git init --bare
```

Common use cases

```
alias gcl  = "git clone --bare "
alias gc   = "git commit -am "
alias gr   = "git remote -v"
alias grso = "git remote show origin"
alias gl1  = "git log --oneline"
alias gwa  = "git worktree add "
alias gt   = "git tag "
alias gpp  = "git pull --rebase "
alias gp   = "git push "
```

git local host or public server with ssh

To upload a Git repository to an SSH server for the first time, follow these steps:

1. Prepare the server for creating the local bare repository on the server(in which will appear remote to each developer.)

Log in to your SSH server and create a bare Git repository (without a working directory) in a suitable location:

```
ssh username@server.com
mkdir -p "/opt/loca/dev/<<project>>"
cd "/opt/loca/dev/<<project>>"
git init --bare
```

This initializes an empty Git repository that can accept pushes.

2. Configure your local repository

On your local machine, initialize a Git repository (if not already done):

```
cd /path/to/your/local/project
git init
git add .
git commit -m "Initial commit"
```

3. Add the remote SSH URL

Set the remote repository URL to point to your server:

```
git remote add origin ssh://username@server.com/opt/loca/dev/<<project>>/
```

4. Push the code

Push your local commits to the remote server:

```
git push -u origin master
```

✅ Note: You must have SSH key authentication set up between your local machine and the server. If not, generate an SSH key pair and add the public key to the server's `~/.ssh/authorized_keys` file.

Optional: Use a Git hook to auto-checkout files If you want the files on the server to be updated automatically (e.g., for deployment,) create a post-receive hook in the server-side repository:

```
cd "/opt/loca/dev/<<project>>/hooks"
nano post-receive
```

Add this script (adjust paths as needed):

```
#!/bin/sh
git --work-tree=/var/www/your-site --git-dir=/opt/loca/dev/<<project>>/ checkout -f
```

Make it executable: `chmod +x post-receive`

Now, every git push will update the live files on the server.

Hooks

Git provides a variety of hooks that trigger at different stages of the Git workflow, categorized into client-side (local repository) and server-side (remote repository) hooks.

Client-Side Hooks

These run on the developer's local machine:

- `pre-commit`: Runs before a commit is made. Used for linting, testing, or formatting checks.
- `prepare-commit-msg`: Runs after the default commit message is created but before the editor opens. Useful for auto-populating messages.
- `commit-msg`: Validates or modifies the commit message after editing. Enforces message standards.
- `post-commit`: Runs after a commit is made. Ideal for notifications or logging.
- `pre-rebase`: Runs before a rebase starts. Can prevent rebasing protected branches.
- `post-checkout`: Runs after git checkout or git switch. Useful for environment setup.
- `post-merge`: Runs after a merge or pull. Can update dependencies or regenerate files.
- `pre-push`: Runs before a push. Used for additional checks or preventing pushes to certain branches.
- `pre-applypatch`: Runs after a patch is applied but before committing.
- `post-applypatch`: Runs after a patch is applied and committed.
- `pre-auto-gc`: Runs before automatic garbage collection.

Server-Side Hooks

These run on the remote repository during network operations:

- `pre-receive`: Runs before updates are accepted. Can reject pushes based on rules.
- `update`: Runs once per ref being updated. Ensures fast-forward-only updates.
- `post-receive`: Runs after all refs are updated. Commonly used for deployment or notifications.
- `post-update`: Runs once after all refs are updated. Often used for updating server-side info.

All hooks are stored in the `.git/hooks directory`. Sample scripts are provided with a `.sample` extension; remove the extension to enable them. Hooks can be written in any executable language (e.g., `shell`, `Python`, `Ruby`.) Use `chmod +x <hook-name>` to make them executable.

For team-wide consistency, use `git config core.hooksPath .githooks` to share hooks via version control.

Original documentation

8.3 Customizing Git - Git Hooks Git Hooks Like many other Version Control Systems, Git has a way to fire off custom scripts when certain important actions occur. There are two groups of these hooks: client-side and server-side. Client-side hooks are triggered by operations such as committing and merging, while server-side hooks run on network operations such as receiving pushed commits. You can use these hooks for all sorts of reasons.

Installing a Hook The hooks are all stored in the hooks subdirectory of the Git directory. In most projects, that's .git/hooks. When you initialize a new repository with git init, Git populates the hooks directory with a bunch of example scripts, many of which are useful by themselves; but they also document the input values of each script. All the examples are written as shell scripts, with some Perl thrown in, but any properly named executable scripts will work fine – you can write them in Ruby or Python or whatever language you are familiar with. If you want to use the bundled hook scripts, you'll have to rename them; their file names all end with .sample.

To enable a hook script, put a file in the hooks subdirectory of your .git directory that is named appropriately (without any extension) and is executable. From that point forward, it should be called. We'll cover most of the major hook filenames here.

Client-Side Hooks

There are a lot of client-side hooks. This section splits them into committing-workflow hooks, email-workflow scripts, and everything else.

Note It's important to note that client-side hooks are not copied when you clone a repository. If your intent with these scripts is to enforce a policy, you'll probably want to do that on the server side; see the example in An Example Git-Enforced Policy.

Committing-Workflow Hooks The first four hooks have to do with the committing process.

The `pre-commit` hook is run first, before you even type in a commit message. It's used to inspect the snapshot that's about to be committed, to see if you've forgotten something, to make sure tests run, or to examine whatever you need to inspect in the code. Exiting non-zero from this hook aborts the commit, although you can bypass it with git commit –no-verify. You can do things like check for code style (run lint or something equivalent), check for trailing whitespace (the default hook does exactly this), or check for appropriate documentation on new methods.

The `prepare-commit-msg` hook is run before the commit message editor is fired up but after the default message is created. It lets you edit the default message before the commit author sees it. This hook takes a few parameters: the path to the file that holds the commit message so far, the type of commit, and the commit SHA-1 if this is an amended commit. This hook generally isn't useful for normal commits; rather, it's good for commits where the default message is auto-generated, such as templated commit messages, merge commits, squashed commits, and amended commits. You may use it in conjunction with a commit template to programmatically insert information.

The `commit-msg` hook takes one parameter, which again is the path to a temporary file that contains the commit message written by the developer. If this script exits non-zero, Git aborts the commit process, so you can use it to validate your project state or commit message before allowing a commit to go through. In the last section of this chapter, we'll demonstrate using this hook to check that your commit message is conformant to a required pattern.

After the entire commit process is completed, the `post-commit` hook runs. It doesn't take any parameters, but you can easily get the last commit by running git log –1 HEAD. Generally, this script is used for notification or something similar.

Email Workflow Hooks

You can set up three client-side hooks for an email-based workflow. They're all invoked by the git am command, so if you aren't using that command in your workflow, you can safely skip to the next section. If you're taking patches over email prepared by git format-patch, then some of these may be helpful to you.

The first hook that is run is `applypatch-msg`. It takes a single argument: the name of the temporary file that contains the proposed commit message. Git aborts the patch if this script exits non-zero. You can use this to make sure a commit message is properly formatted, or to normalize the message by having the script edit it in place.

The next hook to run when applying patches via git am is `pre-applypatch`. Somewhat confusingly, it is run after the patch is applied but before a commit is made, so you can use it to inspect the snapshot before making the commit. You can run tests or otherwise inspect the working tree with

this script. If something is missing or the tests don't pass, exiting non-zero aborts the git am script without committing the patch.

The last hook to run during a git am operation is `post-applypatch`, which runs after the commit is made. You can use it to notify a group or the author of the patch you pulled in that you've done so. You can't stop the patching process with this script.

Other Client Hooks The `pre-rebase` hook runs before you rebase anything and can halt the process by exiting non-zero. You can use this hook to disallow rebasing any commits that have already been pushed. The example pre-rebase hook that Git installs does this, although it makes some assumptions that may not match with your workflow.

The `post-rewrite` hook is run by commands that replace commits, such as git commit –amend and git rebase (though not by git filter-branch). Its single argument is which command triggered the rewrite, and it receives a list of rewrites on stdin. This hook has many of the same uses as the post-checkout and post-merge hooks.

After you run a successful git checkout, the `post-checkout` hook runs; you can use it to set up your working directory properly for your project environment. This may mean moving in large binary files that you don't want source controlled, auto-generating documentation, or something along those lines.

The `post-merge` hook runs after a successful merge command. You can use it to restore data in the working tree that Git can't track, such as permissions data. This hook can likewise validate the presence of files external to Git control that you may want copied in when the working tree changes.

The `pre-push` hook runs during git push, after the remote refs have been updated but before any objects have been transferred. It receives the name and location of the remote as parameters, and a list of to-be-updated refs through stdin. You can use it to validate a set of ref updates before a push occurs (a non-zero exit code will abort the push).

Git occasionally does garbage collection as part of its normal operation, by invoking git gc –auto. The `pre-auto-gc` hook is invoked just before the garbage collection takes place, and can be used to notify you that this is happening, or to abort the collection if now isn't a good time.

Server-Side Hooks

In addition to the client-side hooks, you can use a couple of important server-side hooks as a system administrator to enforce nearly any kind of policy for your project. These scripts run before and after pushes to the server. The pre hooks can exit non-zero at any time to reject the push as well as print an error message back to the client; you can set up a push policy that's as complex as you wish.

`pre-receive` The first script to run when handling a push from a client is pre-receive. It takes a list of references that are being pushed from stdin; if it exits non-zero, none of them are accepted. You can use this hook to do things like make sure none of the updated references are non-fast-forwards, or to do access control for all the refs and files they're modifying with the push.

`update` The update script is very similar to the pre-receive script, except that it's run once for each branch the pusher is trying to update. If the pusher is trying to push to multiple branches, pre-receive runs only once, whereas update runs once per branch they're pushing to. Instead of reading from stdin, this script takes three arguments: the name of the reference (branch), the SHA-1 that reference pointed to before the push, and the SHA-1 the user is trying to push. If the update script exits non-zero, only that reference is rejected; other references can still be updated.

`post-receive` The post-receive hook runs after the entire process is completed and can be used to update other services or notify users. It takes the same stdin data as the pre-receive hook. Examples include emailing a list, notifying a continuous integration server, or updating a ticket-tracking system – you can even parse the commit messages to see if any tickets need to be opened, modified, or closed. This script can't stop the push process, but the client doesn't disconnect until it has completed, so be careful if you try to do anything that may take a long time.

Tip If you're writing a script/hook that others will need to read, prefer the long versions of command-line flags; six months from now you'll thank us.

Git repository types

AI(Artificial Intelligence) vs HI(Human Intelligence?) Or AI with HI?

The primary difference is the presence of a working directory. A non-bare repository has one, allowing for local editing and committing, while a bare repository does not and is used exclusively as a central hub for sharing changes between developers. That said bare repositories can have commites checked out in worktrees in which are amazing for development, where each tree can serve as a feature tree.

Key Differences

Feature	Non-Bare Repository	Bare Repository
Working Directory	Yes (contains your actual project files) yet you are stuck in 1 - one worktree	No (only contains Git metadata,) unless you check out branches into worktrees
Purpose	Local development (writing, editing, and testing code)	Centralized collaboration, sharing, and backup (a "hub",) not to mention a cluster of separate feature trees when developing
Creation	Created by default with git init or git clone (without options)	Created using git init –bare or git clone –bare
Naming Convention	Typically a directory with a hidden .git folder inside	Conventionally ends with .git (e.g., myproject.git)
Direct Pushing	Pushing to the active branch is generally forbidden by default to avoid desynchronization issues	Designed to accept pushes from other repositories without issue

Table 1: The pros and cons of `git --bare`

When to Use Which

- Use a non-bare repository on your local machine where you need to clone, read code, and compile.
- Use a bare repository on your local machine where you intend to implement features in separate worktrees, edit files, stage changes, and create commits(your standard development workflowi.)
- Use a bare repository on a server to act as a central remote repository (like the ones hosted on your own server w/ssh access; web or otherwise, GitHub, GitLab, or Bitbucket.) Developers clone this repository to their local machines, work in their non-bare clones, and then push their changes back to the bare remote.

My amendments from the AI response in this color.

git bare vs not

AI-oversikt

The primary difference is the presence of a **working directory**. A non-bare repository has one, allowing for local editing and committing, while a bare repository does not and is used exclusively as a central hub for sharing changes between developers. Stack Overflow +3

Key Differences

Feature	Non-Bare Repository	Bare Repository
Working Directory	Yes (contains your actual project files)	No (only contains Git metadata)
Purpose	Local development (writing, editing, and testing code)	Centralized collaboration, sharing, and backup (a "hub")
Creation	Created by default with `git init` or `git clone` (without options)	Created using `git init --bare` or `git clone --bare`
Naming Convention	Typically a directory with a hidden `.git` folder inside	Conventionally ends with `.git` (e.g., `myproject.git`)
Direct Pushing	Pushing to the active branch is generally forbidden by default to avoid desynchronization issues	Designed to accept pushes from other repositories without issue

When to Use Which

- **Use a non-bare repository** on your local machine where you need to edit files, stage changes in the index, and create commits (your standard development workflow).
- **Use a bare repository** on a server to act as a central remote repository (like the ones hosted on GitHub, GitLab, or Bitbucket). Developers clone this repository to their local machines, work in their non-bare clones, and then push their changes back to the bare remote. Reddit +4

Figure 2: This is what AI answered when asking about the pros and cons of `git --bare`

Cloning a public project using –bare option

```
% git clone --bare https://github.com/TheAppgineer/roon-tui
Cloning into bare repository 'roon-tui.git'...
remote: Enumerating objects: 692, done.
remote: Counting objects: 100% (42/42), done.
```

```
remote: Compressing objects: 100% (21/21), done.
remote: Total 692 (delta 26), reused 26 (delta 21), pack-reused 650 (from 1)
Receiving objects: 100% (692/692), 351.57 KiB | 3.06 MiB/s, done.
Resolving deltas: 100% (370/370), done.
cd roon-tui.git
```

Inspecting the bare repo after adding the master branch as a worktree

```
git worktree add master
% eza -lT
drwxr-xr-x@    - user 12 Mar 14:40 .
.rw-r--r--@  177 user 12 Mar 14:37 ├── config
.rw-r--r--@   73 user 12 Mar 14:37 ├── description
.rw-r--r--@   23 user 12 Mar 14:37 ├── HEAD
drwxr-xr-x@    - user 12 Mar 14:37 ├── hooks
.rwxr-xr-x@  478 user 12 Mar 14:37 │   ├── applypatch-msg.sample
.rwxr-xr-x@  896 user 12 Mar 14:37 │   ├── commit-msg.sample
.rwxr-xr-x@ 4.7k user 12 Mar 14:37 │   ├── fsmonitor-watchman.sample
.rwxr-xr-x@  189 user 12 Mar 14:37 │   ├── post-update.sample
.rwxr-xr-x@  424 user 12 Mar 14:37 │   ├── pre-applypatch.sample
.rwxr-xr-x@ 1.6k user 12 Mar 14:37 │   ├── pre-commit.sample
.rwxr-xr-x@  416 user 12 Mar 14:37 │   ├── pre-merge-commit.sample
.rwxr-xr-x@ 1.4k user 12 Mar 14:37 │   ├── pre-push.sample
.rwxr-xr-x@ 4.9k user 12 Mar 14:37 │   ├── pre-rebase.sample
.rwxr-xr-x@  544 user 12 Mar 14:37 │   ├── pre-receive.sample
.rwxr-xr-x@ 1.5k user 12 Mar 14:37 │   ├── prepare-commit-msg.sample
.rwxr-xr-x@ 2.8k user 12 Mar 14:37 │   ├── push-to-checkout.sample
.rwxr-xr-x@ 2.3k user 12 Mar 14:37 │   ├── sendemail-validate.sample
.rwxr-xr-x@ 3.6k user 12 Mar 14:37 │   └── update.sample
drwxr-xr-x@    - user 12 Mar 14:37 ├── info
.rw-r--r--@  240 user 12 Mar 14:37 │   └── exclude
drwxr-xr-x@    - user 12 Mar 14:40 ├── master
.rw-r--r--@  40k user 12 Mar 14:40 │   ├── Cargo.lock
.rw-r--r--@  921 user 12 Mar 14:40 │   ├── Cargo.toml
drwxr-xr-x@    - user 12 Mar 14:40 │   ├── images
.rw-r--r--@ 4.9k user 12 Mar 14:40 │   │   ├── save-preset.png
.rw-r--r--@  79k user 12 Mar 14:40 │   │   ├── screenshot.png
.rw-r--r--@ 7.0k user 12 Mar 14:40 │   │   ├── unicode-symbols.png
.rw-r--r--@ 9.6k user 12 Mar 14:40 │   │   └── zone-selection.png
.rw-r--r--@ 1.1k user 12 Mar 14:40 │   ├── LICENSE
.rw-r--r--@ 9.8k user 12 Mar 14:40 │   ├── README.md
drwxr-xr-x@    - user 12 Mar 14:40 │   └── src
drwxr-xr-x@    - user 12 Mar 14:40 │       ├── app
.rw-r--r--@  36k user 12 Mar 14:40 │       │   ├── mod.rs
.rw-r--r--@ 4.7k user 12 Mar 14:40 │       │   ├── stateful_list.rs
.rw-r--r--@  32k user 12 Mar 14:40 │       │   └── ui.rs
drwxr-xr-x@    - user 12 Mar 14:40 │       ├── io
.rw-r--r--@  898 user 12 Mar 14:40 │       │   ├── events.rs
.rw-r--r--@ 1.6k user 12 Mar 14:40 │       │   ├── mod.rs
.rw-r--r--@  41k user 12 Mar 14:40 │       │   └── roon.rs
.rw-r--r--@ 1.1k user 12 Mar 14:40 │       ├── lib.rs
.rw-r--r--@ 3.0k user 12 Mar 14:40 │       └── main.rs
drwxr-xr-x@    - user 12 Mar 14:37 ├── objects
drwxr-xr-x@    - user 12 Mar 14:37 │   ├── info
drwxr-xr-x@    - user 12 Mar 14:37 │   └── pack
.r--r--r--@  20k user 12 Mar 14:37 │       ├── pack-
a14c26322da12eff829b29fc85910abcea56d6aa.idx
```

```
.r--r--r--@ 360k user 12 Mar 14:37 │       ├── pack-
a14c26322da12eff829b29fc85910abcea56d6aa.pack
.r--r--r--@ 2.8k user 12 Mar 14:37 │       └── pack-
a14c26322da12eff829b29fc85910abcea56d6aa.rev
.rw-r--r--@ 1.4k user 12 Mar 14:37 ├── packed-refs
drwxr-xr-x@    - user 12 Mar 14:37 ├── refs
drwxr-xr-x@    - user 12 Mar 14:40 │   ├── heads
drwxr-xr-x@    - user 12 Mar 14:37 │   └── tags
drwxr-xr-x@    - user 12 Mar 14:40 └── worktrees
drwxr-xr-x@    - user 12 Mar 14:40     └── master
.rw-r--r--@    6 user 12 Mar 14:40         ├── commondir
.rw-r--r--@   49 user 12 Mar 14:40         ├── gitdir
.rw-r--r--@   23 user 12 Mar 14:40         ├── HEAD
.rw-r--r--@ 1.6k user 12 Mar 14:40         ├── index
drwxr-xr-x@    - user 12 Mar 14:40         ├── logs
.rw-r--r--@  170 user 12 Mar 14:40         │   └── HEAD
.rw-r--r--@   41 user 12 Mar 14:40         ├── ORIG_HEAD
drwxr-xr-x@    - user 12 Mar 14:40         └── refs
```

Inspecting the master worktree

```
cd master
% eza -lT
drwxr-xr-x@    - user 12 Mar 14:40 .
.rw-r--r--@  40k user 12 Mar 14:40 ├── Cargo.lock
.rw-r--r--@  921 user 12 Mar 14:40 ├── Cargo.toml
drwxr-xr-x@    - user 12 Mar 14:40 ├── images
.rw-r--r--@ 4.9k user 12 Mar 14:40 │   ├── save-preset.png
.rw-r--r--@  79k user 12 Mar 14:40 │   ├── screenshot.png
.rw-r--r--@ 7.0k user 12 Mar 14:40 │   ├── unicode-symbols.png
.rw-r--r--@ 9.6k user 12 Mar 14:40 │   └── zone-selection.png
.rw-r--r--@ 1.1k user 12 Mar 14:40 ├── LICENSE
.rw-r--r--@ 9.8k user 12 Mar 14:40 ├── README.md
drwxr-xr-x@    - user 12 Mar 14:40 └── src
drwxr-xr-x@    - user 12 Mar 14:40     ├── app
.rw-r--r--@  36k user 12 Mar 14:40     │   ├── mod.rs
.rw-r--r--@ 4.7k user 12 Mar 14:40     │   ├── stateful_list.rs
.rw-r--r--@  32k user 12 Mar 14:40     │   └── ui.rs
drwxr-xr-x@    - user 12 Mar 14:40     ├── io
.rw-r--r--@  898 user 12 Mar 14:40     │   ├── events.rs
.rw-r--r--@ 1.6k user 12 Mar 14:40     │   ├── mod.rs
.rw-r--r--@  41k user 12 Mar 14:40     │   └── roon.rs
.rw-r--r--@ 1.1k user 12 Mar 14:40     ├── lib.rs
.rw-r--r--@ 3.0k user 12 Mar 14:40     └── main.rs
```

Create new worktree based on prior knowledge.

```
% git worktree add newfeature
Preparing worktree (new branch 'newfeature')
HEAD is now at 3159297 Merge pull request #20 from TheAppgineer/19-update-time-
dependency
% du -sh *
4.0K    config
4.0K    description
4.0K    HEAD
 64K    hooks
4.0K    info
308K    master
```

```
308K    newfeature
376K    objects
4.0K    packed-refs
4.0K    refs
 48K    worktrees
```

Create new empty worktree for adding features that do not require prior knowledge.

```
% git worktree add --no-checkout newemptytree
```

When you create an empty worktree using the orphan branch method, you are actually creating a new branch (with no parent) and checking it out into a separate working directory. The worktree itself is just a folder where that branch's files live; the branch exists independently in the repository. Here's how you can understand and verify that relationship.

1. The concept: branch vs. worktree

Branch: A named pointer to a commit in the Git repository. It lives in the repository's database (the .git folder, or in your case, the bare repository /project.git).

Worktree: A directory on your filesystem where a particular branch is checked out, allowing you to work on that branch without affecting others.

When you run:

```
git checkout --orphan feature-empty
```

you create a new branch named feature-empty that has no commit history yet. Then you add files and make the first commit, which becomes the branch's root commit.

When you later add a worktree for that branch:

```
git worktree add ~/project/feature-empty feature-empty
```

you are simply telling Git to create a working directory (/project/feature-empty) where that branch is checked out. The branch already exists.

2. How to know that your empty worktree "becomes" a branch

After creating the orphan branch and making a commit, you can verify the branch exists and is associated with the worktree in several ways.

a) From within the worktree Navigate to the worktree directory and use:

```
cd ~/project/feature-empty

# Show the current branch
git branch --show-current
# Output: feature-empty

# Or see branch info in status
git status
# On branch feature-empty
# nothing to commit, working tree clean
```

b) List all branches in the repository From any worktree (or even from the bare repo directory), run:

```
git branch -v
```

This shows all local branches and the latest commit on each. You should see feature-empty listed alongside main.

c) List all worktrees and their associated branches

```
git worktree list
```

Example output:

text /project/main main [abc1234] commit message /project/feature-empty feature-empty [def5678] commit message This tells you exactly which branch is checked out in each worktree.

d) View the branch's commit history

```
git log --oneline feature-empty
```

This shows the commits that belong only to that branch. If you created the branch from scratch (orphan), its history will be independent.

3. Why the worktree feels "empty" initially

When you first switch to an orphan branch (e.g., with git checkout –orphan), Git removes all tracked files from the working directory and staging area, leaving you with a completely clean slate. After you add files and commit, the branch now has content. Any new worktree you add for that branch will start with that committed content, not empty.

If you want a truly empty worktree at the moment of creation, you would:

Create the orphan branch and make an initial commit without any files (i.e., an empty commit).

```
git checkout --orphan feature-empty
git commit --allow-empty -m "root commit"
```

Then add a worktree for it:

```
git worktree add ~/project/feature-empty feature-empty
```

Now the worktree will contain no files (except for possible Git metadata like .git), because the branch's commit is empty.

4. Summary

The branch is created when you use `--orphan` (or `-b` with a new name) and make the first commit.

The worktree is just a checkout of that branch in a separate folder.

You can always verify which branch is associated with a worktree using `git branch --show-current` or `git worktree list`.

So the empty worktree doesn't "become" a branch; rather, it is the working copy of a branch that you created. The branch is the abstract pointer; the worktree is where you interact with its files.